Chapter 8 lecture script: Numerical optimisation

Draft for the course author's review. **Target: 150 minutes in total, with 116 minutes of slides on 31 slides (including two discussions, the three practice questions and one live demo of 18 minutes), 14 minutes of slack and two breaks of 10 minutes.** The breaks separate the three blocks of the lecture. The only code that is run live is the demo of slide 22, from the script `demo-diagnosis.R`.

This script follows the Chapter 8 theory from the introduction to the recipe, with a short introduction to the R practical. The chapter has no new data: each difficulty is shown on a model that students fitted in an earlier chapter. The main thread is the complete-pooling model of the CO_2 uptake of 12 plants from Chapter 7, which stops in several wrong places from plausible starting values; the reed frog functional response of Chapter 6 shows a search on a surface and estimates that run to a boundary; the owl calls and the fir seedlings of Chapter 4 show the effect of parameterisation and a search in one dimension. The lecture covers how an optimiser searches and when it stops, how to diagnose a fit, multiple starting values, identifiability and boundaries, scaling and parameterisation, other local optimisers and global search. The slides carry keywords only; the narrative is here.

Slide 3 uses your diagram `ModelCycle.png` from the Chapter 1 lecture folder. Slides 5 (`co2-stops.png`), 7 (`search-paths.png`) and 10 (`ridge-slice.png`) use figures in `lecture-assets/`. All three are unchanged copies of Chapter 8 figures: `fig-co2_stops`, `fig-search_paths` and `fig-ridge_slice`. Slide 6 uses `minima-schematic.svg`, a schematic without numerical scales made for the lecture. The code of all four is in `lecture-assets/figures.R`, which runs the chapter's chunks in the chapter's order (none of them draws random numbers) and writes the schematic. If the chapter figures change, run that script again from the repository root; it takes a few seconds.

Timing and slide sequence

Times are measured from the start of the session and include the breaks.

Slide	Topic	Minutes	Elapsed time
1	Ecological Models and Data Analysis	1	0–1
2	Overview of the lecture	2	1–3
3	Where we are: the modelling cycle	1	3–4
4	What an optimiser does	3	4–7
5	A fit that stops in the wrong place	4	7–11
6	Local minima, saddle points and ridges	3	11–14
7	How a gradient method searches	4	14–18
8	Exact and approximate gradients	3	18–21
9	Stopping rules and convergence codes	3	21–24

Slide	Topic	Minutes	Elapsed time
10	Diagnosing a fit	4	24–28
11	Practice 1: three fits of a parasitoid	3	28–31
12	Discussion: has this fit found the maximum?	4	31–35
	Break	10	35–45
13	The Hessian at the end of a search	3	45–48
14	Multiple starting values	4	48–52
15	Starting values in a model with random effects	3	52–55
16	Structural and practical non-identifiability	4	55–59
17	Estimates at a boundary	4	59–63
18	Practice 2: identifiability of seed predation	3	63–66
19	Scaling	3	66–69
20	Parameterisation	3	69–72
21	Practice 3: scaling a growth curve	3	72–75
	Break	10	75–85
22	Live demo: diagnosing a failed fit	2 + 16	85–103
23	Other local optimisers	4	103–107
24	One parameter: a search within an interval	2	107–109
25	Comparing optimisers fairly	3	109–112
26	Global search: differential evolution	5	112–117
27	Global search, then local refinement	5	117–122
28	Discussion: which remedy for which symptom?	4	122–126
29	A recipe for a difficult fit	3	126–129
30	The Chapter 8 R practical	2	129–131
31	Summary and closing questions	5	131–136
	Slack for discussion	14	136–150
	Total	**150**	

Slides 116 minutes (blocks of 35, 30 and 51, the demo included), slack 14, breaks 20: 150 minutes. The break before the demo lets you open R and run section 0 of `demo-diagnosis.R`.

Slides 1 and 2 — Ecological Models and Data Analysis; Overview of the lecture

In this lecture, we learn how to find the maximum of a likelihood when the optimiser does not find it by itself, and how to tell whether it has. Since Chapter 4 every fit has minimised a negative log-likelihood (NLL) with `optim()`, the BFGS method and the gradient that RTMB supplies, and from the starting values that we chose this has usually worked. An optimiser can, however, stop somewhere other than the maximum and still report a convergence code of zero. The practical of Chapter 4 met this with a Gamma model of penguin body masses whose fit stopped after a single step, and the practical of Chapter 7 with fits that stopped at a practically zero variance among sites although the maximum lay at a positive variance.

The lecture is organised by problem, as the chapter is. Each part states a difficulty in general terms, shows it on a fit from an earlier chapter that goes wrong, explains as much of the mechanism as we need, and gives the remedy. BFGS with the gradient of RTMB remains the method of the course; the other optimisers are the next tool when it fails.

The session has three blocks, separated by two breaks. The first block shows a fit that stops in the wrong place, explains how a gradient method searches, what the gradient costs, when the search stops and how to diagnose a fit, and ends with a practice question and a discussion. The second turns to the Hessian at the end of a search, multiple starting values, identifiability, boundaries, scaling and parameterisation. The third starts with a live demo that diagnoses a failed fit and repairs it, then shows other local optimisers and global search, and ends with the recipe, the practical and the summary.

Delivery

Read the overview at the pace of the list, one keyword and its question at a time, and do not explain any term yet.

Slide 3 — Where we are: the modelling cycle

This lecture looks inside one entry of the estimation box: “optimise (4 and 8)”. Every model of the course, from the Poisson model of the fir seedlings to the mixed model of the CO_2 uptake, was fitted by handing an NLL to an optimiser, and every estimate, standard error, likelihood profile and comparison of models since Chapter 4 assumed that the optimiser had found the maximum. This chapter asks what happens when it has not, and it completes the line on estimation that started with the sampling distributions of Chapter 3.

Delivery

One minute. Point to “optimise (4 and 8)” in the estimation box, and say that the other boxes do not change in this lecture.

Slide 4 — What an optimiser does

An **optimiser** is a search for the parameter values $\boldsymbol{\theta}$ that minimise the NLL, $-\ell(\boldsymbol{\theta})$, starting from values that we give it. It searches locally. At each point that it evaluates, it knows the value of the NLL and its **gradient** $\mathbf{g}(\boldsymbol{\theta})$, the vector of partial derivatives of the NLL with respect to the

parameters, and nothing else. It never sees the whole surface that we drew for two parameters in Chapter 4. From these local quantities it decides where to go next, takes a step, and repeats, until a stopping rule tells it to end (slide 9). It returns the estimates, the minimum of the NLL that it reached and a convergence code.

The call in the example box is the one we have used since Chapter 4. `MakeADFun()` processes the NLL; `obj$par` holds the starting values on the optimised scale, `obj$fn` evaluates the NLL and `obj$gr` its gradient, which RTMB calculates exactly. `optim()` with `method = "BFGS"` returns the estimates in `par`, the minimum in `value`, the counts of evaluations in `counts` and the code in `convergence`.

The maximum likelihood estimate is the global minimum of the NLL: the lowest point of the whole surface. Because the optimiser sees only the points it visits, where it ends depends on where it started and on the shape of the surface along the way. That is why the takeaway matters: what we want is the global minimum, and what the optimiser gives us is the end of one search.

Delivery

Point at the three arguments of the call and ask what each one gives the optimiser (starting values, the value, the slope). The slides that follow use these names.

Slide 5 — A fit that stops in the wrong place

The same model, fitted from different starting values, can end in different places, each with convergence code 0. To show it, the chapter returns to the complete-pooling model of the CO_2 uptake of Chapter 7. The data come from Potvin, Lechowicz and Tardif (1990), "The statistical analysis of ecophysiological response curves obtained from experiments involving repeated measures", *Ecology* 71: 1389–1400, and are built into R as `CO2`: the CO_2 uptake (μmol/m²/s) of 12 plants of the grass *Echinochloa crus-galli*, each measured at seven ambient concentrations of CO_2 from 95 to 1000 μL/L, 84 measurements in all. The model describes all of them with one saturating exponential, $\mu(x) = a\left(1 - e^{-b(x-c)}\right)$, with the asymptote a (μmol/m²/s), the rate b (L/μL) and the concentration c (μL/L) at which the expected uptake is zero, and with a Normal distribution of the measurements around the curve. The plants are ignored, which is the simplification of complete pooling; here the model serves only as a fit that can go wrong.

Chapter 7 read its starting values from a figure: an asymptote of 30 μmol/m²/s, zero uptake at 50 μL/L, a rate of 0.01 L/μL and a standard deviation of 2 μmol/m²/s. From these values BFGS reaches the maximum, with an NLL of 296.26. The chapter then fits the same object from three other starting values, each a plausible reading of the same figure. A slow rise ($b = 0.001$ L/μL) fits the plants that are still rising at the highest concentration. A high asymptote (100 μmol/m²/s) assumes that the curve is far from levelling off. An early plateau (a fast rate of 1 L/μL, zero uptake at 90 μL/L and an asymptote of 10 μmol/m²/s) fits the chilled plants from Mississippi, which level off almost at once. All three fits report code 0, and none reaches the maximum.

Figure walkthrough

The blue points are the 84 measurements. The solid vermillion curve is the end of the fit from the starting values of Chapter 7, the maximum. The dashed line is the end of the fit from the slow

rise: the rate ran to about 5×10^{11} L/µL, so the curve jumps to its asymptote just above c and is flat over all the measured concentrations. It is the model without a curve: its asymptote, 27.2 µmol/m²/s, is the mean of all 84 measurements (NLL 318.68). The dotted line at zero is the end of the fit from the high asymptote: the asymptote and the rate both ran towards zero, the curve predicts no uptake at all, and a standard deviation of 29.3 µmol/m²/s absorbs the whole size of the measurements (NLL 402.79). The dot-dashed curve is the end of the fit from the early plateau: c stopped at 90.3 µL/L, just below the lowest concentration, and the curve rises steeply below 95 µL/L and is nearly flat above it (NLL 302.32). Ask students which curve they would believe from the figure alone; most will pick the solid one, which is the point: the code does not tell them, the figure and the NLL do. The figure cannot show why each search stopped; slides 7 to 10 explain it.

Slide 6 — Local minima, saddle points and ridges

A few kinds of points matter for where a search can end. A **local minimum** of the NLL is lower than all the points near it. The **global minimum** is the lowest point of all, and it is the maximum likelihood estimate. A **stationary point** is a point where the gradient is zero: a minimum, a maximum, or a **saddle point**, where the NLL rises in some directions and falls in others. A **ridge** is a direction along which the likelihood, and so the NLL, hardly changes; on the optimised scale (a logarithm, a logit) it often runs towards $\pm\infty$. An optimiser can stop at a local minimum, near a saddle point, and on a ridge, where the gradient is practically zero although the point is not a stationary point in the strict sense.

The three wrong stops of slide 5 are of these kinds. The constant curve lies on a ridge: once the rate is large enough, making it larger changes nothing. The curve at zero lies on another, where the asymptote and the rate run towards zero together. The stop near $c = 90$ µL/L has a small gradient, but the NLL falls in one direction, a combination of the location and the rate (slide 13).

Figure walkthrough

The schematic shows the NLL against one parameter, without scales. The dip on the left is a local minimum, the deeper dip in the middle the global minimum, and the flat stretch on the right a ridge, on which a search finds no slope to follow. A saddle point cannot be drawn with one parameter: it needs at least two, like a mountain pass, which is a minimum along the path over the pass and a maximum along the ridge that it crosses. Point out that a search that starts on the left ends in the local minimum, and one that starts on the right stays on the ridge.

Slide 7 — How a gradient method searches

A **gradient method** moves downhill. The negative gradient points in the direction in which the NLL falls fastest, so the simplest method takes a step in that direction, evaluates the NLL and the gradient at the new point, and repeats. How far to step is decided by a **line search**: the method tries a step of some length along the chosen direction and shortens it until the NLL falls by enough. Following the steepest slope alone is slow in a narrow, tilted valley, such as the surface of correlated estimates in Chapter 4, because the steepest direction points across the valley and not along it.

Newton's method also uses the curvature. Near a minimum the NLL is close to a parabola, the quadratic approximation of Chapter 4, and Newton's method jumps to the minimum of the parabola that has the gradient and the Hessian of the current point. With one parameter the step is $-g/H$, the slope divided by the curvature; with several it is $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \mathbf{H}(\boldsymbol{\theta}_t)^{-1}\mathbf{g}(\boldsymbol{\theta}_t)$, where $\mathbf{H}$ is the Hessian, the matrix of second derivatives. Close to the minimum this is fast. Far from it the parabola can be a poor description: the jump can be far too long, and where the curvature is negative in some direction, the parabola has no minimum and the step can go uphill.

BFGS, the method we have used since Chapter 4, is a **quasi-Newton** method: it takes steps like those of Newton's method, but with an approximation of the curvature that it builds from the gradients of successive steps. Two nearby gradients that differ show how the slope changes, which is what the curvature measures. So BFGS needs only `fn` and `gr`, and it becomes about as fast as Newton's method close to a minimum. In `optim()`, its first step goes along the negative gradient, because it has not yet learned any curvature. The method is named after Broyden, Fletcher, Goldfarb and Shanno, who published it independently in 1970; Nocedal and Wright (2006), *Numerical Optimization*, 2nd edition, Springer, describe it in detail. The update formula is in the chapter's box "Statistical detail: Newton's method and BFGS"; students do not need it.

Newton's method still has a place in the course: RTMB uses it for the inner optimisation of a model with random effects in Chapter 7, where automatic differentiation supplies the exact second derivatives and the search starts close to the minimum (slide 15).

Whatever the method, the consequence is the takeaway of the list: a gradient method follows the slope from wherever it starts, so it ends at the minimum, ridge or boundary to which the slope leads.

Figure walkthrough

The figure is the likelihood surface of the reed frog functional response of Chapter 6, a Binomial model with the Holling type II curve and two parameters, the attack rate α and the handling time H. The grey contours are the relative log-likelihood, drawn on the original scales, and the vermillion circle is the maximum. BFGS searches $\text{logit}\,(\alpha)$ and $\log(H)$, so the right edge of the figure ($\alpha = 1$) and the bottom edge ($H = 0$) lie at $+\infty$ and $-\infty$ on the scales that it searches. The points are where BFGS evaluated the gradient, the start and the end of each accepted step, and the lines join them in order. From the starting values of Chapter 6 ($\alpha = 0.5$, $H = 1/60$) the search reaches the maximum in five steps. From $\alpha = 0.01$ the first step goes along the steep gradient and is so long that α jumps to practically 1, and the search ends there, on the edge, with an NLL of 59.77 against 46.72 at the maximum. From $\alpha = 0.05$, $H = 0.03$ the search first runs close to the same edge and then jumps to H practically 0, where it ends at 52.22. Slide 17 returns to these two fits. The figure cannot show that both report code 0; the chapter's table does.

Slide 8 — Exact and approximate gradients

When `optim()` is not given `gr`, it approximates the gradient. Each partial derivative is replaced by a **finite difference**, the change in the NLL between two nearby values of one parameter divided by their distance, as in Chapter 5. `optim()` uses central differences, one evaluation on each side

of the current value, so each approximate gradient of a model with p parameters costs $2p$ extra evaluations of the NLL. The distance between the two values is a compromise: a large distance makes the straight line between the two points a poor approximation of the slope at the point, and a very small one makes the difference of two nearly equal numbers inaccurate, because the computer rounds every number it stores. The default step is 10^{-3} for every parameter (the control argument `ndeps`; details in the chapter's box "Technical detail: finite differences"). The gradient that RTMB calculates by automatic differentiation (Chapter 6) has neither problem: it is exact up to rounding error, and it costs about as much as a few evaluations of the NLL, whatever the number of parameters.

For the CO_2 model from the starting values of Chapter 7, the fit without `gr` also reports code 0, but it ends slightly above the minimum: 296.2565 against 296.2557. The element `counts` of the result does not include the evaluations for the finite differences, as the help page of `optim()` says. With four parameters each approximate gradient costs eight of them, so the fit without `gr` called `fn` $83 + 8 \times 32 = 339$ times, against 104 calls to `fn` and 33 to `gr` with the exact gradient. Here the approximation costs time and a little precision; in a model with many parameters, or in a narrow valley, it can stop the search well before the minimum.

The takeaway has two halves. The exact gradient helps, and it is one reason why the course writes every NLL for RTMB. But from the three other starting values of slide 5, BFGS without `gr` stops in the same wrong places as with it: the failures of slide 5 are not caused by an inaccurate gradient.

Delivery

If students ask why `counts` reports 32 evaluations of `gr` when no `gr` was given: `optim()` counts the approximate gradients it calculated, not the calls to `fn` inside them.

Slide 9 — Stopping rules and convergence codes

A **stopping rule** decides when an optimiser ends its search. The rule of BFGS in `optim()` looks at the value of the NLL, not at the gradient. A step that lowers the NLL by less than `reltol` $\times$ $(|-\ell(\boldsymbol{\theta})| +$ `reltol`$)$ counts as no progress. BFGS then discards its approximation of the curvature and tries a step along the negative gradient; when that step makes no progress either, the search stops and `optim()` returns convergence code 0. The tolerance `reltol` is $\sqrt{\epsilon} \approx 1.5 \times 10^{-8}$ by default, where $\epsilon \approx 2.2 \times 10^{-16}$ is the precision of numbers in R. The search also stops after `maxit` iterations, 100 by default for BFGS, and then the code is 1. The help page of `optim()` lists other codes, which belong to other methods. This description was checked against the source of `optim()` (the routine `vmmin` in R's `src/appl/optim.c`).

The tolerance is relative: it scales with the size of the NLL. For the CO_2 model, whose NLL is about 300, a step must lower it by more than about 4.4×10^{-6} to count as progress. For the Adelie body masses of the practical, in grams, the NLL is about 1138, so the threshold is about 1.7×10^{-5}; the first step of the fit lowered the NLL by 1.64×10^{-5}, just less, and because the first step of BFGS already goes along the negative gradient, the search stopped at once. Exercise 2 of the practical takes this up.

In plain English, BFGS stops when it can no longer lower the NLL, and it does not check the gradient at all. At a minimum this is what we want. The same thing happens, however, on a ridge, where the NLL hardly changes in any direction that the search tries; near a saddle point, where the gradient is close to zero; at a boundary on the optimised scale, where the NLL keeps falling by ever smaller amounts as a parameter runs towards $\pm\infty$; and on a surface that is nearly flat compared with the size of the NLL. So a convergence code of 0 says that the stopping rule was met: the search stopped making progress. Whether it reached the maximum is decided by the checks of slide 10. This is the point of the chapter's one warning callout, and it is the takeaway of slide 5; it does not need repeating on every later slide.

Chapter 4's note "When a fit fails" dealt with a code other than zero and with an NLL that cannot be calculated. This chapter deals with a code of zero that hides a failure.

Check for understanding

Ask: "Would a smaller `reltol` cure the stops of slide 5?" Expected answer: not the ridge and not the curve at zero, where the NLL no longer falls in the directions that the search tries; a stricter tolerance only adds iterations there (checked with `reltol` of 10^{-12}: 318.68 and 402.79 again). The early plateau is different: the NLL still falls slowly along its negative curvature (slide 13), and with `reltol` of 10^{-12} BFGS continues from it to the minimum of 296.26. That is a reason to diagnose a fit, not to tighten the tolerance, because we cannot know in advance which kind of stop we have. The course never sets `reltol`; the remedies are a restart, multiple starts, rescaling, reparameterisation and another optimiser.

Slide 10 — Diagnosing a fit

Diagnosing a fit means checking whether the point where the optimiser stopped is the maximum of the likelihood. Each check looks at the fit from a different side, and none of them is sufficient alone:

1. **The estimates on the original scale.** Are they sensible values in their units? Is a parameter very large or very small on the optimised scale (a logarithm of 27 or of -245), which means that it ran towards a boundary or along a ridge?
2. **The gradient at the solution**, `obj$gr(fit$par)`, whose elements should be close to zero. A large element shows that the search stopped on a slope. A small one is necessary but not enough, because the gradient is also close to zero on a ridge and near a saddle point.
3. **The curvature**, from `sdreport()`. At a minimum the Hessian is positive definite (slide 13), and the element `pdHess` of the result is `TRUE`. When it is not, the standard errors are `NaN` and printing the result gives the warning "Hessian of fixed effects was not positive definite". Huge standard errors are a warning sign as well (slide 16). By default `sdreport()` uses the parameter values with the lowest NLL that the object has evaluated, so after several fits of the same object we pass the values of the fit to check with the argument `par.fixed`.
4. **A refit**, from the solution and from other starting values. A refit that starts where the fit stopped and moves on shows that the search stopped too early; fits from other starts that end lower show that it stopped at the wrong minimum.

5. **A slice** through the solution along a suspicious parameter, with the other parameters fixed at the solution: the likelihood slice of Chapter 4, now used as a diagnostic. It shows how the NLL behaves beyond the point where the optimiser stopped.

For the four CO_2 fits, the fit from the starting values of Chapter 7 passes every check: a small gradient (0.0025 at its largest), a positive definite Hessian, and a refit that stays where it is. The fits that ended at the constant curve and at zero have even smaller gradients, so the gradient does not reveal them, but `sdreport()` does: for the constant curve the largest gradient is 5×10^{-6}, but `pdHess` is `FALSE` and none of the standard errors can be calculated. At that point the rate is so large that the curve is flat over all the measured concentrations, and changing the rate or the location no longer changes the NLL: these are flat directions, which have no curvature. The fit at zero is flat in three directions for the same reason.

Figure walkthrough

The figure is a slice through the end of the fit from the slow rise: the asymptote, the location and the standard deviation are fixed where the fit stopped, and the rate varies from 0.001 to 10 L/µL on a logarithmic axis. The vertical axis is the relative log-likelihood, the difference from the log-likelihood where the fit stopped, and the dashed line marks zero. For rates above about 0.2 L/µL the curve is already flat over the measured concentrations, and the slice is a plateau at zero. The fit stopped far to the right of the plot, at about 5×10^{11} L/µL, where the slope is zero and BFGS had no direction in which to move. At about 0.015 L/µL the slice rises about 12 units above the plateau, so the point where the fit stopped is not the maximum even along this one line. The slice also shows how the search got there: once a step took the rate onto the plateau, nothing led it back. What the slice cannot show is the best value of the other parameters at each rate; that would be a likelihood profile.

Slide 11 — Practice 1: three fits of a parasitoid

Short activity — allow about two minutes of work and one to discuss

Students work on paper. The setting, which the slide shortens: a parasitoid wasp was offered patches with 2 to 40 hosts, and the number of eggs it laid in each patch was counted; a colleague describes the expected number of eggs with the Michaelis–Menten function $ax/(b + x)$ of the number of hosts, with a Poisson model, fitted with BFGS on the scales $\log(a)$ and $\log(b)$ from three sets of starting values. a is in eggs per patch and b in hosts per patch. Expected answers, from the chapter's solution:

1. **Fit A** is the best candidate: the lowest NLL, code 0, a small gradient, a positive definite Hessian and sensible estimates (a maximum of about 41 eggs per patch, half of it reached at about 19 hosts). Three fits are few, so a refit from A and a few more starting values would confirm it.
2. **Fit B** reached the limit of iterations (code 1). Its NLL is only 0.03 above that of A and its estimates are close, but its gradient is not small: the search was still moving when it was stopped. A restart from where B stopped would most likely end at A.
3. **Fit C** reports code 0 and has a tiny gradient, but its NLL is 16.6 units higher than that of A, both parameters are enormous, and the Hessian is not positive definite. The ratio a/b is about 2.1 eggs per host, and when b is much larger than every x, the curve $ax/(b + x)$ is practically the

straight line $(a/b)\,x$. So C stopped on a ridge that leads to the linear model without saturation, where a and b can grow together without changing the NLL. It is reported as a failed fit, not as estimates.

Students who pick C because of its code and its gradient have relied on the two checks that slide 10 calls insufficient. Students who reject B only because of its code are right that it is not the reported fit, but should see that it almost reached A.

Slide 12 — Discussion: has this fit found the maximum?

The three fits on the slide are three of the CO_2 fits of slide 5, without the curves: the fit from the starting values of Chapter 7, the constant curve and the stop near $c = 90$ µL/L. The takeaway lists the five checks of slide 10, which students apply.

Discussion and expected answers — included in the four minutes

Give students two minutes to decide with a neighbour, then take one answer per fit.

1. **NLL 296.3, gradient 0.003, `pdHess TRUE`, sensible estimates.** This is the fit to report, after a refit and a few more starting values. It passes every check, and its NLL is the lowest of the three, which alone rules out the other two as the maximum.
2. **NLL 318.7, gradient 5×10^{-6}, `pdHess FALSE`, rate 5×10^{11}.** The estimates and the check of the Hessian decide it. A rate of 5×10^{11} L/µL is absurd, and the Hessian has flat directions; the gradient is the smallest of the three and does not help. A slice along the rate shows the plateau of slide 10, and the NLL is 22.4 units above fit 1. It is the ridge of the constant curve.
3. **NLL 302.3, gradient 0.02, `pdHess FALSE`, one negative curvature.** The Hessian decides it: a negative curvature means that the NLL falls in some direction, so the point is not a minimum. The gradient is small and a refit from this point stays where it is, so those two checks pass. The NLL is 6.1 units above fit 1. Slide 13 explains this case after the break.

Accept "compare the NLLs" as the first check for all three: fit 1 is lower than the others, so they cannot be the maximum. Push students to name the check that would reveal fits 2 and 3 if fit 1 were not there, because in practice we do not know the lowest NLL in advance.

Break (10 minutes).

Slide 13 — The Hessian at the end of a search

The **Hessian** $\mathbf{H}(\boldsymbol{\theta})$ is the matrix of second derivatives of the NLL. It describes the curvature of the NLL in every direction: for a direction $\mathbf{d}$, a vector of changes in the parameters, the curvature of the NLL along the straight line through the point in that direction is proportional to $\mathbf{d}^\top \mathbf{H}\,\mathbf{d}$. The Hessian is **positive definite** when this curvature is positive in every direction, that is, when the NLL curves upwards whichever way we move. A point with a zero gradient and a positive definite Hessian is a local minimum, and the inverse of the Hessian is the covariance matrix of Chapter 4, from which `sdreport()` calculates the standard errors.

The curvatures along a set of perpendicular directions are the **eigenvalues** of the Hessian, and the Hessian is positive definite when all of them are positive. A zero eigenvalue is a flat direction, as on a ridge: the NLL does not change to second order along it, the Hessian has no inverse,

and the standard errors are `NaN`. A negative eigenvalue is a direction in which the NLL curves downwards, so the point is not a minimum; if the gradient is also zero there, the point is a saddle point. This is what `pdHess` checks: that the NLL rises in every direction from the solution, which is what a minimum requires. The details are in the chapter's box "Statistical detail: the Hessian at a minimum".

The stop of the CO_2 model at $c = 90.3$ µL/L is the case of a negative curvature. Its gradient is small (0.019 at its largest) and a refit stays where it is. Of the four curvatures of its Hessian, three are clearly positive and one is slightly negative, along a direction that combines the location and the rate: if both change together, the NLL falls. A slice along c alone would show a minimum at 90.3 µL/L, because the direction in which the NLL falls is not along one parameter. That is the takeaway: the Hessian looks in every direction at once, while a slice looks along one.

Delivery

Connect to slide 7: at a point with a negative curvature, the parabola of Newton's method has no minimum, which is why the quadratic approximation of Chapter 4 only describes the NLL near a minimum. If students ask how to see the eigenvalues: the chapter checks them in a hidden chunk; for their own fits, `pdHess` and the standard errors from `sdreport()` are the check.

Slide 14 — Multiple starting values

The end of a local search depends on its starting values. Good starting values, read from a figure or calculated with the method of moments, as in Chapters 4 to 7, usually suffice. The CO_2 model shows that the choice matters more than it might seem: the start from the slow rise ends on the ridge with a standard deviation of 2 µmol/m²/s, but the same start with a standard deviation of 10 µmol/m²/s reaches the maximum, because the search then follows a different path. When a fit is in doubt, or a model is known to be difficult, we use **multiple starting values**, in the four steps of the slide:

1. choose a box of plausible values for each parameter, in its units, and transform it to the optimised scale;
2. draw starting values at random within the box and fit the model from each;
3. keep the fit with the lowest NLL, and check how many fits reach it and whether their estimates agree;
4. restart BFGS from the best fit, and diagnose it as on slide 10.

Drawing the starting values uniformly on the optimised scale gives each order of magnitude of a positive parameter the same weight, which suits parameters such as rates whose plausible values span several orders of magnitude. The loop is a `for` loop over `optim()` with the seed set once, before the loop, so that the starting values can be reproduced.

For the CO_2 model the box is an asymptote between 5 and 100 µmol/m²/s, a rate between 10^{-4} and 1 L/µL (so that the curve reaches 63% of its asymptote between 1 and 10,000 µL/L above c), a location between 0 and 90 µL/L (below the lowest concentration) and a standard deviation between 0.5 and 20 µmol/m²/s, with 50 starts and seed 801. Seventeen of the 50 fits reach the minimum of 296.3, and their estimates agree: the location, which varies most, lies between 45.0

and 46.8 µL/L. Another 17 end on the ridge of the constant curve (318.7) and 14 at the curve of zero (402.8), all with code 0; one reached the limit of iterations at 298.7 (code 1), and one ended far away. Without the first fit of the chapter we would still have found the maximum, and the table tells us which failures to expect from this model. A restart of BFGS from the best of the 50 fits ends at the same minimum as the fit of Chapter 7, with a positive definite Hessian. Students will run this loop in the demo (slide 22).

The takeaway is the one caveat of multiple starts: agreement among many starts is evidence that the global minimum was found, but it cannot prove it, because a minimum that no start reached does not appear in the results.

Check for understanding

Ask: "Why draw the rate uniformly on the log scale, and not between 10^{-4} and 1 L/µL directly?" Expected answer: uniformly on the original scale, nine in ten starts would have a rate above 0.1, and the slow rates would hardly be tried; on the log scale each factor of 10 gets the same share.

Slide 15 — Starting values in a model with random effects

A model with random effects has the same failures. Its NLL is the marginal NLL of Chapter 7, in which the random effects are integrated out with the Laplace approximation, so the fixed effects form a surface with the same ridges and early plateaus as before. Each evaluation of the marginal NLL, however, contains an inner optimisation: for given values of the fixed effects, RTMB finds the values of the random effects that maximise the joint likelihood, with Newton's method (slide 7), which can use the exact second derivatives of automatic differentiation. Every fit of a mixed model is therefore much slower than a fit of the corresponding model without random effects, and multiple starts cost more.

The model with a random asymptote of Chapter 7 reaches the minimum of the marginal NLL, 207.61, from the starting values of Chapter 7. From the slow rise (an asymptote of 30 µmol/m²/s, a rate of 0.001 L/µL and a location of 50 µL/L) it stops with code 0 at 307.72, with the rate running towards infinity; from an asymptote of 100 µmol/m²/s, a rate of 1 L/µL and a location of 90 µL/L it stops with code 0 at 261.17, with the location at 91.6 µL/L, just below the lowest concentration. These are the ridge and the early plateau of the model without plants, and multiple starting values are the remedy here too. A variance among groups at zero is a further failure that only grouped models have; slide 17 and Exercise 3 of the practical return to it.

Delivery

Keep this slide short if the block runs late (see "Adjustments during delivery"). The point to keep is that nothing in this lecture is specific to models without random effects.

Slide 16 — Structural and practical non-identifiability

A parameter is **identifiable** when different values of it give different distributions of the data, so that the likelihood can tell them apart. When a parameter is not identifiable, the NLL does not change, or hardly changes, along some direction in the space of the parameters, and the optimiser has nothing to guide it along that direction. There are two kinds.

Structural non-identifiability is a property of the model: the parameters enter the distribution of the data only through a combination of them, such as their product or their sum, so that different values give exactly the same distribution, whatever the data. The NLL is then exactly flat along a ridge, and the estimates depend on the starting values. **Practical non-identifiability** is a property of the model and the data together: the model could tell the values apart, but the data cannot. Chapter 5 showed that the observations that can tell us about a parameter are those made where the curve responds to it; when the data contain no such observations, the NLL is nearly flat in the direction of that parameter.

Both kinds show through the checks of slide 10: estimates that stay close to their starting values, or that differ between fits from different starts with practically the same NLL; very large standard errors, or standard errors that are `NaN`; correlations between estimates close to 1 or -1; the warning of `sdreport()` that the Hessian is not positive definite, which is typical of structural non-identifiability; and a slice or a likelihood profile that is flat over a wide range. The remedies depend on the kind. A structurally non-identifiable model must be changed: a parameter is removed or fixed, or the model is reparameterised so that only the combination that the data determine is estimated. A practically non-identifiable model may be fine, and more informative data would solve the problem: a design whose observations cover the range where the curve responds to the parameter. When neither is possible, the parameter is reported as not determined by the data, with its likelihood profile, and not as an estimate with a standard error.

The chapter has one example of each. The structural one is a model between the no-pooling and the partial-pooling models of Chapter 7: a common log asymptote plus a deviation for each of the 12 plants, without a distribution for the deviations. It has 13 parameters for 12 asymptotes, because adding any number to the common log asymptote and subtracting it from every deviation gives the same asymptotes. Fitted from common asymptotes of 10, 20 and 40 μmol/m²/s, all three fits reach 171.44, the minimum of the no-pooling model, and agree on the asymptote of every plant, but the common asymptote ends at a different value in each fit and none of the standard errors can be calculated. One of the three fits passes the check of the Hessian, because the zero curvature along the flat direction is zero only up to rounding error; the agreement of the NLL and of the plant asymptotes, with a common asymptote that depends on the start, is the reliable symptom. Removing the common asymptote gives the no-pooling model; giving the deviations a Normal distribution, as Chapter 7 did, ties the common asymptote to the centre of the plant asymptotes and makes it identifiable.

The practical one uses the complete-pooling model fitted only to the 60 measurements at 250 μL/L and above, without the low concentrations at which the curve responds to the location c. The fit reports code 0 and the Hessian is positive definite, so `sdreport()` gives no warning, but c stays at its starting value of 50 μL/L with a standard error of 579 μL/L, against 21.7 μL/L with all 84 measurements, and the estimates of the log rate and the location have a correlation of 0.992. Over the concentrations from 250 to 1000 μL/L, a curve that reaches zero far to the left and rises slowly looks like one that reaches zero at 50 μL/L and rises faster. The likelihood profile of c stays within 0.4 units of its highest value from -1000 to 245 μL/L, far less than the cutoff of 1.92, so the interval has no lower end; with all the data the interval runs from -26.6 to 75.0 μL/L. With

the reduced data we would report that the location is not determined, and the remedy is a design with measurements at low concentrations.

Delivery

Make clear that the ridge of slide 5 is a different case: with all 84 measurements the location is determined, and the ridge leads towards the constant curve, whose NLL is 22.4 units above the minimum. It is a direction that the data do not favour, on which a search can stop.

Slide 17 — Estimates at a boundary

The maximum of a likelihood, or a point to which an optimiser is drawn, can lie at the **boundary** of the range of a parameter: a probability of 0 or 1, or a variance or a handling time of zero. On the optimised scale, the logarithm of a positive parameter or the logit of a probability, the boundary lies at $-\infty$ or $+\infty$. The optimiser then walks towards it with ever smaller decreases of the NLL, and stops with code 0 when the decreases fall below the tolerance of slide 9. The estimate on the optimised scale is a large number that depends on where the search happened to stop, and its standard error cannot be calculated, because the NLL is flat out there. That is the takeaway: such an estimate is reported as being at the boundary (an attack rate of 1, a handling time of 0), or the model is changed; the number on the optimised scale is not reported.

The two reed frog searches of slide 7 are examples. The search from a small attack rate stopped with logit (α) at about 204, an attack rate of 1 to the precision of the computer, and the search from the third start with $\log(H)$ at about -16, a handling time of 9×10^{-8} (in units of the length of the experiment per tadpole killed), practically zero. Both report code 0, and neither passes the check of the Hessian. Neither is the maximum, because the NLL at the maximum is lower (46.72 against 59.77 and 52.22): these are boundaries towards which the searches were drawn from poor starting values, and multiple starts would reveal them as failures.

A boundary can also be where the maximum lies. A variance among groups whose maximum likelihood estimate is zero is the common case in grouped models: it happens when the groups differ less than the variation within groups would make them differ by chance. On the scale of the log standard deviation the maximum then lies at $-\infty$, and BFGS stops on its way there with code 0. The estimate is reported as zero, which means that the model reduces to complete pooling, and the likelihood profile of the standard deviation (Chapter 7) shows which values the data allow. A fit that stops at a practically zero variance can also be a failure, with the maximum at a positive variance; Exercise 3 of the practical tells the two cases apart.

Some optimisers accept **bounds**: `optim(method = "L-BFGS-B")` and `nlminb()` take the arguments `lower` and `upper` on the scale of the parameters, so that a search can reach a boundary exactly instead of walking towards infinity. The course keeps the transformations of Chapter 4 instead, because then every value on the optimised scale is allowed, and the standard errors, likelihood profiles and `ADREPORT()` all work on a scale without edges. An estimate at a boundary is recognised by a parameter that runs towards $\pm\infty$ on the optimised scale.

Slide 18 — Practice 2: identifiability of seed predation

Short activity — allow about two minutes of work and one to discuss

Students work on paper. The setting: rodents are offered trays with 5 to 40 seeds for one night, and the number of seeds removed from each tray is counted, with a Poisson distribution of the counts. In the first model x is the number of seeds in the tray, a the proportion of seeds that rodents find in one night and p the probability that a seed that is found is removed; in the second, earlier studies suggest a half-maximum b of about 500 seeds per tray. Expected answers, from the chapter's solution:

1. $\boldsymbol{\lambda = a\,p\,x}$ is structurally non-identifiable: a and p enter the mean only through their product ap, the proportion of the seeds that are removed, so any pair with the same product gives the same distribution of the counts. A fit would reach the same NLL from different starting values with different estimates of a and p, with `NaN` standard errors and, usually, the warning that the Hessian is not positive definite. The remedy is to estimate the product as one parameter, or to add data that measure one of the two separately (for example, observations of how many of the seeds that rodents find they remove).
2. $\boldsymbol{\lambda = ax/(b + x)}$ **with** $\boldsymbol{b}$ **near 500** is identifiable in principle, but all the trays have far fewer seeds than the half-maximum, where the curve is practically the straight line $(a/b)\,x$. The data determine the initial slope a/b and not a and b separately: practical non-identifiability. A fit would show very large standard errors, a correlation of the estimates of $\log(a)$ and $\log(b)$ close to 1, flat likelihood profiles, or a search that runs along the ridge towards the straight line, as fit C of Practice 1 (slide 11). The remedy is a design with trays around and above 500 seeds, or the linear model with the slope a/b as its only parameter of the curve.

Point out the connection to slide 11: fit C of the parasitoid was the same ridge, met from the side of the optimiser.

Slide 19 — Scaling

The same model can be easy or difficult to fit depending on how its parameters are written. Two properties matter: **scaling**, the magnitudes of the parameters on the optimised scale (this slide), and **parameterisation**, the shape of the surface (slide 20). Neither changes the maximum or the fitted model, by the invariance of Chapter 4, but both change the path of the search.

An optimiser works best when a change of about one unit in each parameter changes the NLL by similar amounts. Its steps must suit every parameter at once, and the first step of BFGS goes along the negative gradient, with the same step length for all of them; a step that is right for an asymptote of 30 is far too large for a rate of 0.01. There are three remedies. The first is a transformation: on the log scale a change of one unit multiplies a parameter by e, whatever its size, so a rate of 0.01 and an asymptote of 30 become -4.6 and 3.4. The second is the choice of units for the data and for untransformed parameters; the units also set the size of the NLL, which matters for the relative tolerance of slide 9 (the penguin masses in grams of the practical). The third is the control argument `parscale` of `optim()`, which gives the typical size of each parameter: `optim()` then searches over the parameters divided by these sizes, so that every parameter it works with is of about one unit.

For the CO_2 model with the rate on its original scale, fitted from the starting values of Chapter 7, the search ends on the ridge of the constant curve, with code 0, an NLL of 318.68 and a rate of 1.27

L/µL, far out on the plateau of slide 10. The gradient at the starting values shows why: its element for the rate is 100 times that for the log rate, because by the chain rule the derivative with respect to $\log(b)$ is b times the derivative with respect to b, and $b = 0.01$ L/µL. So the first step changes the rate far more, relative to its size, than the log rate would change, and it takes the rate onto the plateau, from which nothing leads back. With `parscale = c(1, 0.01, 10, 1)`, which measures the rate in units of 0.01 L/µL and the location in units of 10 µL/L, BFGS reaches the minimum of 296.26, as it does with the log rate of Chapter 7.

A transformation is the usual remedy in the course, because it also deals with the constraint that the rate is positive. `parscale` is the tool for a parameter that cannot be transformed, such as the location c, whose scale is set by its units.

Slide 20 — Parameterisation

A **parameterisation** changes the shape of the surface and, with it, the correlation of the estimates (Chapter 4). Strongly correlated estimates mean a narrow valley that is tilted with respect to the axes of the parameters, along which the NLL changes slowly and across which it changes fast. Such a valley is harder to search, because the steepest direction points across it (slide 7), and the approximation of the curvature that BFGS builds must learn the tilt before its steps can follow it. This is one reason why the course writes distributions with their mean where possible: the data usually determine the mean well, and its estimate is often nearly uncorrelated with those of the other parameters.

For the owl calls of Chapter 4 (the visits of the female parent to nests with satiated nestlings), the Negative Binomial model can be written with the mean μ and the shape k, or with the shape and the probability p (`size` and `prob` of `dnbinom()`). Both fits from the method-of-moments estimates reach the same minimum, as invariance requires, and the correlation of the estimates is practically zero with μ and k and 0.71 with k and p. Here the tilt does not hurt BFGS, which needs a similar number of evaluations in both cases (16 of `fn` and 5 of `gr`, against 14 and 6). A correlation of 0.71 is a mild tilt. A narrower valley can stop a search early: the Gamma model of the Adelie masses in the practical has a correlation of 0.996 between the estimates of the log shape and the log rate, and BFGS stops after one step there; with the mean and the shape the correlation is practically zero and the fit reaches the maximum. The reed frog model of Chapter 6 lies in between, with a correlation of 0.82 for logit (α) and $\log(H)$, and of 0.98 in the Michaelis–Menten form with $\log(a)$ and $\log(b)$.

Delivery

Say honestly that the owl example does not show a failure: the slide shows what a parameterisation changes (the tilt), and the practical shows a case where the tilt matters.

Slide 21 — Practice 3: scaling a growth curve

Short activity — allow about two minutes of work and one to discuss

Students use R as a calculator. The setting: the length of a fish (mm) as a function of its age x in days, with a Normal distribution of the lengths around the curve; the asymptote, the rate and

the standard deviation are optimised on the log scale and the location c untransformed. Expected answers, from the chapter's solution:

1. With age in days the starting values on the optimised scale are $\log(600) \approx 6.40$, $\log(0.0005) \approx -7.60$, $c = -100$ and $\log(20) \approx 3.00$. In R: `c(log(600), log(0.0005), -100, log(20))`.
2. The location is poorly scaled: it is -100, while the other three lie between -8 and 7, and a step of one unit is a change of one day in c, which hardly changes the curve, but a factor of e in a, b or σ. With age in years the rate is multiplied by 365 and the location divided by 365: $\log(0.1825) \approx -1.70$ and $c \approx -0.27$, so all four values are of comparable size. In R: `c(log(600), log(0.0005 * 365), -100 / 365, log(20))`.

The callout in the chapter also asks whether a change of the units of length helps. It does not: a log-transformed parameter only shifts by a constant when its units change (by $\log(10)$ from millimetres to centimetres), so its scale stays the same; the units of length change only the size of the NLL. Mention it if time allows.

Break (10 minutes).

Slide 22 — Live demo: diagnosing a failed fit

The demo runs the steps of slides 5 to 14 on one fit: the CO_2 model from the slow rise, which stops on the ridge of the constant curve. It diagnoses the fit, repairs it with multiple random starting values, refines and confirms the best fit, and draws it. It has no comparison of optimisers; those follow on slides 23 to 27. The script needs the packages RTMB, RTMBdist and TMB (for `TMB::tmbprofile()`); the data set `CO2` is built into R. Its code and object names are those of the chapter, so the numbers agree with the chapter and the slides. The whole script runs in a few seconds (1.5 seconds in a check in R 4.6.1), so there is no waiting.

During the break, open R and run section 0, "Packages, data and the NLL". It loads RTMB and RTMBdist (with their messages about masked functions), builds `co2` and the list of data `co2_data`, draws the 84 measurements in base R, defines `pooled_nll()` and `cmb()`, and processes the NLL with `MakeADFun()` from the starting values of the slow rise (an asymptote of 30 µmol/m²/s, a rate of 0.001 L/µL, zero uptake at 50 µL/L and a standard deviation of 2 µmol/m²/s). Keep the session and the plot open. Spend the two minutes at the slide reading the five steps, then switch to R and run the sections one at a time.

Walkthrough of the script — 16 minutes

1. **A fit from a poor start (about 3 minutes).** Fit with BFGS and the gradient of RTMB. The convergence code is 0 and the minimum of the NLL 318.68. On the original scale the asymptote is 27.21 µmol/m²/s, the rate 5.2×10^{11} L/µL, the location 49.9 µL/L and the standard deviation 10.75 µmol/m²/s. Ask the first question below. The plot draws the curve of the fit (dashed) and the mean uptake (dotted) over the data: the two lines coincide over the measured concentrations, because the curve is the constant at the mean of the 84 measurements, and the standard deviation is that of the measurements around their mean.
2. **Diagnosis (about 4 minutes).** `pooled_obj$gr(poor_fit$par)`: the largest element is 5.2×10^{-6}, and the elements for the rate and the location are exactly zero, so the gradient does not

reveal the problem. `sdreport()` with `par.fixed`: `pdHess` is `FALSE`, every standard error is `NaN`, and printing the result gives the warning that the Hessian of the fixed effects was not positive definite. Say why `par.fixed` is needed here (slide 10) only if a student asks. The slice along the log rate, with the other parameters fixed at the solution, is drawn as the relative log-likelihood against the rate on a logarithmic axis: the plot of slide 10 in base R, flat to the right and about 12 units higher at 0.015 L/µL. Ask the second question below.

3. **Multiple random starts (about 4 minutes).** Read the box in units before running it: asymptote 5 to 100 µmol/m²/s, rate 10^{-4} to 1 L/µL, location 0 to 90 µL/L, standard deviation 0.5 to 20 µmol/m²/s, transformed to the optimised scales. Point at the seed, set once before the loop, and at the line that draws one set of four starting values with `runif()` in each repetition. The table of where the fits ended: 17 at 296.3, 17 at 318.7, 14 at 402.8, all with code 0; one at 298.7 with code 1; and one at 2098.8, far away. Point at the 318.7 row: the failure of section 1 is one of the two common failures of this model.
4. **Refine and confirm (about 4 minutes).** Restart BFGS from the best of the 50 fits: 296.2557, the minimum of the chapter. `sdreport()`: `pdHess` is `TRUE`, and the standard errors on the optimised scales are 0.044 (`log_asym`), 0.31 (`log_rate`), 21.7 µL/L (`c0`) and 0.077 (`log_sd_obs`), with a largest gradient of 0.003. The likelihood profile of the location with `TMB::tmbprofile()` and `confint()`: the 95% interval runs from −26.6 to 75.0 µL/L. `confint()` warns that it collapses repeated values of the profile; the warning is harmless, as the comment in the script says. The plot of the profile object is TMB's own base R plot: it draws the NLL (a valley, not a peak) against the location, with a dotted horizontal line 1.92 above the minimum and dotted vertical lines at the ends of the interval. The interval is wide, because the data say little about the concentration at which the uptake reaches zero (slide 16), but it has two ends: the location is determined.
5. **The fitted curve (about 1 minute).** The curve of the best fit (solid, vermillion) and the curve of the poor start (dashed) over the data. On the original scale the best fit has an asymptote of 32.5 µmol/m²/s, a rate of 0.0095 L/µL, a location of 45.9 µL/L and a standard deviation of 8.23 µmol/m²/s, the estimates of Chapter 7. End by naming the four steps that the demo followed: symptom, diagnosis, remedy, confirmation, which are the four steps of every exercise of the practical (slide 30).

Questions to ask during the demo (about 1 minute, within the sections above)

1. In section 1, "Is a rate of 5×10^{11} L/µL an estimate?" No: it is a parameter that ran towards $+\infty$ on the log scale, and its value is where the search happened to stop. The curve is flat over the data for any rate above about 0.2 L/µL.
2. In section 2, "The gradient is practically zero. Why is this not the maximum?" Because the gradient is also zero on a ridge (slide 10): the NLL does not change along the rate, so the slope is zero, and the slice shows a higher log-likelihood elsewhere along the same line.
3. In section 3, "Seventeen of 50 fits reached 296.3. Does that prove that it is the global minimum?" No: it is evidence, because many starts from a wide box agree, but a minimum that no start reached would not appear in the table (slide 14).

Slide 23 — Other local optimisers

BFGS with the gradient of RTMB is the method of the course, and the remedies of the previous slides usually make it work. When it keeps failing, another optimiser is the next tool. Every optimiser below takes `fn` of the same object of `MakeADFun()`, and `gr` if it uses a gradient, so the NLL does not change. The script names the packages and routines; the slide keeps to the general points.

`nlminb()` calls the optimisers of the PORT library. Like BFGS, it uses the gradient and an approximation of the curvature (or the Hessian itself, if it is given), but it controls its steps differently: it trusts the local parabola only within a certain distance of the current point, a **trust region**, and adjusts that distance as the search goes. It is the optimiser of the examples in the documentation of RTMB and TMB, and glmmTMB, which the supplement on formula-based methods introduces, uses it by default. Its arguments are `start`, `objective` and `gradient` (`obj$par`, `obj$fn` and `obj$gr`), and its result has the elements `par`, `objective` (the minimum of the NLL), `convergence`, `message` (why it stopped) and `evaluations`. From the three starting values of slide 5 at which BFGS stopped in the wrong place, `nlminb()` reaches the minimum of 296.2557 from all three: a search that controls its steps differently avoids the ridge, the curve at zero and the early plateau from these starts.

The **Nelder–Mead** method, `optim(method = "Nelder-Mead")`, the default of `optim()`, uses no gradient at all. With p parameters it keeps a **simplex** of $p + 1$ points (a triangle for two parameters) and the NLL at each of them. At each step it replaces the worst point by its reflection through the centre of the others; if the reflected point is much better it tries a point further out (an expansion), and if it is worse it tries one closer in (a contraction); when nothing helps, the whole simplex shrinks towards its best point. It stops when the values of the NLL at the points of the simplex agree within the relative tolerance. Because it needs no gradient, it suits an NLL that cannot be differentiated, but it needs many evaluations, and it stops less precisely than a gradient method.

From the 50 random starts of slide 14, drawn again with the same seed in the same order, 17 fits of BFGS, 42 of `nlminb()` and 12 of Nelder–Mead end within 0.01 of the minimum, in this example. The other 8 fits of `nlminb()` stop at the ridge (318.7). Nelder–Mead ends close to the minimum from many starts (34 within 2 units) but reaches it exactly from fewer, because it stops when the points of its simplex have similar values, which can happen a few tenths of a unit above the minimum; three of its 50 fits also stopped at its limit of 500 iterations, with code 1. Each of these fits would be recognised in a diagnosis, and a restart of BFGS from the best of them would end at the minimum. “In this example” matters: slide 25 explains why these counts are not a ranking.

Delivery

If students ask whether the course should switch to `nlminb()`: the course keeps BFGS, which works for every model of the chapters when the starting values are reasonable; `nlminb()` is a good second opinion, and glmmTMB users meet it anyway.

Slide 24 — One parameter: a search within an interval

For a single parameter, `optimize()` searches an interval that we give it. It combines **golden-section search**, which shrinks the interval around the minimum by a fixed fraction at each step, with parabolic interpolation through three points, which jumps to the minimum of the parabola when the NLL is smooth. It needs no gradient and no starting value, only the interval on the optimised scale. Its arguments are `f`, the NLL as a function of one number, and `interval`, the lower and upper ends of the search; it returns `minimum`, the estimate, and `objective`, the minimum of the NLL. Nelder–Mead is not the alternative here: in one dimension `optim()` warns that it is unreliable and recommends `optimize()`.

For the fir seedlings of Chapter 4, 50 quadrats with a Poisson model on the log scale, `optimize()` searches $\log(\lambda)$ between -3 and 3, that is, between 0.05 and 20 seedlings per quadrat. The estimate is 2.14 seedlings per quadrat, the mean of the counts, as Chapter 4 showed for the Poisson model, and the search needed no starting value.

Slide 25 — Comparing optimisers fairly

Comparing optimisers fairly means giving them the same NLL and the same starting values, comparing the minimum of the NLL that each reaches and not only the estimates (two fits with similar estimates can differ in the NLL, and the NLL is what the maximum likelihood estimate is defined by), and counting the evaluations of `fn` and `gr`, which are the cost. Even then, one model is not a ranking: another model, or another set of starting values, can reverse the order. The counts of slide 23 come from one model, one box and one seed; they say which optimiser helped there, not which is better in general. That is the takeaway: compare what each optimiser reaches on your problem, not how it is described.

Check for understanding

Ask: "A colleague reports that Nelder–Mead is better than BFGS because its estimates looked more sensible in one fit. What is missing?" Expected answer: the NLLs of both fits, the same starting values, the number of evaluations, and more than one model or start.

Slide 26 — Global search: differential evolution

Every method so far is **local**: it follows one path from its starting values. A **global** method explores a whole box of parameter values instead. **Differential evolution** keeps a **population** of parameter vectors, drawn at random within the box, and improves it over many **generations**. In each generation, every member of the population gets a challenger. The challenger is made by adding a scaled difference of two other members to a third member (**mutation**), and by taking each of its parameters either from this mutant or from the current member (**crossover**). The challenger replaces the member if its NLL is lower (**selection**). Differences between members are large while the population is spread over the box and shrink as it gathers around a minimum, so the steps adapt to the surface without a gradient.

In symbols, the classical mutant for member i is $\mathbf{v}_i = \mathbf{x}_{r_0} + F\left(\mathbf{x}_{r_1} - \mathbf{x}_{r_2}\right)$, where $\mathbf{x}_{r_0}$, $\mathbf{x}_{r_1}$ and $\mathbf{x}_{r_2}$ are three other members chosen at random and F is a weight, usually between 0 and 1; in the crossover each element comes from the mutant with probability CR. `DEoptim()` uses $F = 0.8$ and

$CR = 0.5$ by default, and its default strategy also moves each mutant part of the way towards the best member of the population. With NP members and G generations the search evaluates the NLL $NP \times (G + 1)$ times. The method is due to Storn and Price (1997), "Differential evolution: a simple and efficient heuristic for global optimization over continuous spaces", *Journal of Global Optimization* 11: 341–359; the R package DEoptim is described by Mullen, Ardia, Gil, Windover and Cline (2011), *Journal of Statistical Software* 40(6): 1–26.

The last bullet is the cost. The method needs a box, which sets where it can look; thousands of evaluations; and a seed, because it is random. It can find a minimum that local searches miss, but it proves nothing: the population can gather around a local minimum as well, and it approaches the minimum it finds only slowly. Other global methods, such as simulated annealing and genetic algorithms, work on similar principles; the course does not use them.

For the CO_2 model, `DEoptim::DEoptim()` with the box of slide 14, a population of 40 and 200 generations (seed 802) reaches the minimum of 296.26 after $40 \times 201 = 8,040$ evaluations of the NLL, against about 100 for one fit of BFGS from good starting values.

Slide 27 — Global search, then local refinement

A global search finds the region of the minimum without starting values, and a local search finds the minimum precisely. So the two are used together: **global, then local**. BFGS from the best member of the population is precise near the minimum, and its result is diagnosed like any other fit, with the gradient, `sdreport()` and, if needed, a slice or a profile. That is the takeaway: global search, then BFGS, then diagnosis.

The code shows the two calls. `DEoptim::DEoptim()` takes `fn`, the NLL (`obj$fn`); `lower` and `upper`, the box on the optimised scale, which the method needs; and `control`, made with `DEoptim::DEoptim.control()`, with the size of the population `NP`, the number of generations `itermax` and `trace = FALSE` to stop it printing its progress. The elements of its result that we use are in its element `optim`: `bestmem`, the best member of the final population; `bestval`, its NLL; and `nfeval`, the number of evaluations. The course calls DEoptim with its prefix and does not attach the package. The second line starts BFGS from `bestmem`.

For the CO_2 model, BFGS from the best member stops almost at once at 296.2557, with a positive definite Hessian, and its estimates agree with those of the fit from the starting values of Chapter 7. Here differential evolution found the maximum likelihood estimates without any starting values, at the cost of many evaluations. For a model with random effects, in which every evaluation contains an inner optimisation (slide 15), that cost grows quickly, and multiple starts of BFGS, or of `nlminb()`, are usually cheaper.

Delivery

The code on the slide is shortened. In the chapter the elements of `bestmem` are named `par1` to `par4` by `DEoptim()`, and the chapter gives them the names of `pooled_obj$par` before the BFGS fit, so that the result is labelled. Mention it if a student copies the code.

Slide 28 — Discussion: which remedy for which symptom?

This discussion asks students to match the symptoms of the lecture with the remedies. The takeaway lists the five remedies in the order of the symptoms, so the task is to justify the match.

Discussion and expected answers — included in the four minutes

Give students two minutes with a neighbour, then take one answer per item.

1. **A parameter of 10^{-250} on the original scale, standard errors `NaN`.** A boundary or a ridge: the parameter ran towards $-\infty$ on the log scale (slide 17). Decide whether it is a failure (other starts reach a lower NLL, as for the curve at zero of slide 5, whose log asymptote was about -245) or the maximum at a boundary (a variance of zero), and report it at the boundary or change the model.
2. **Different NLLs from different starts, all with code 0.** Multiple starts (slide 14): keep the lowest, count how many fits reach it, restart BFGS from it and diagnose it.
3. **A rate of 0.003 and an asymptote of 3,000, and an early stop.** Scaling (slide 19): fit both on the log scale, change the units, or use `parscale` for a parameter that cannot be transformed.
4. **A standard error of 579 for a parameter measured in tens.** Identifiability (slide 16): most likely practical non-identifiability, as for the location of the CO_2 curve without low concentrations. Draw the likelihood profile; change the design, or report the parameter as not determined.
5. **BFGS keeps stopping at wrong places after every remedy so far.** Another optimiser: `nlminb()` with the same object, and as a last resort a global search with `DEoptim::DEoptim()` followed by BFGS from its best member (slides 23 and 27).

Accept other matches when students justify them: item 1 can be structural non-identifiability if the parameter moves with another one, and item 2 can reveal a ridge or a boundary once the fits are diagnosed. The point is that each symptom points first to a diagnosis, and the remedy follows from it.

Slide 29 — A recipe for a difficult fit

The steps of this chapter extend the recipes of Chapters 6 and 7 at the step of the fit, and the chapter collects them in a note callout before the summary. Read the eight steps with the tool of each: check that the NLL returns an ordinary number at the starting values, with `obj$fn(obj$par)`, and that its gradient `obj$gr(obj$par)` has no `NaN`; fit with BFGS and the gradient of RTMB, and check the convergence code; diagnose the fit with the estimates on the original scale, the gradient at the solution, `sdreport()` (`pdHess` and the standard errors), a refit from the solution and a slice along a suspicious parameter; if the fit is in doubt or the model is known to be difficult, fit it from multiple starting values within a box of plausible values, keep the lowest NLL, check how many fits reach it and restart BFGS from it; if a parameter runs along a ridge or to a boundary, decide whether it is identifiable, and change the model or the data, or report the parameter as not determined or at the boundary; if the parameters have very different magnitudes or highly correlated estimates, rescale or reparameterise; if BFGS still fails, try `nlminb()` with the same object and, as a last resort, a global search with `DEoptim::DEoptim()` followed by BFGS from its best member; and report

what was done: the starting values, the failures and how they were treated, and the diagnosis of the final fit.

Delivery

About two minutes. Steps 1 and 2 are what students have done since Chapter 4; point at the new ones, 3 to 8. Step 8 is the one that students forget: a fit that needed multiple starts is reported with the box, the seed and the number of starts that reached the minimum.

Slide 30 — The Chapter 8 R practical

The practical returns to three fits of earlier practicals that go wrong. Exercise 1, on the growth of 14 loblolly pines (`Loblolly`, built into R), is worked on the page: the growth curve with complete pooling of the Chapter 7 practical, fitted from a start with $k = 0.1$/year that is as plausible as the start of Chapter 7, stops at a curve that is practically zero at every age; the exercise diagnoses it with the gradient, `sdreport()` and a slice, repairs it with 20 random starts, `nlminb()` and differential evolution followed by BFGS, and confirms the estimates of Chapter 7. Exercise 2 returns to the Gamma model of the body masses of Adelie penguins (`penguins`, built into R from version 4.5.0) of the Chapter 4 practical, whose fit in grams stops after one step: the stopping rule of slide 9, the tilted valley of slide 20, and three remedies (kilograms, the mean and the shape, `nlminb()`). Exercise 3 takes the REML fits of the simulation of a variance among sites from the Chapter 7 practical, some of which stop at a practically zero variance, and tells apart the fits whose maximum is at zero from those that missed a positive maximum. Exercise 1 uses the package DEoptim, which students install once.

Each exercise follows the four steps of the takeaway: the symptom, the diagnosis, the remedy and the confirmation. Try each exercise before checking the solution.

Slide 31 — Summary and closing questions

An optimiser searches locally: it knows only the value and the gradient of the NLL at the points that it evaluates, and it follows the slope from its starting values. BFGS takes steps like those of Newton's method, with an approximation of the curvature built from successive gradients, and the exact gradient of RTMB makes it more precise and cheaper than finite differences. Its stopping rule looks at the decrease of the NLL relative to its size, so a convergence code of 0 says that the search stopped making progress, which happens at a minimum but also on a ridge, near a saddle point, at a boundary and on a nearly flat surface. Whether a fit found the maximum is decided by a diagnosis: the estimates on the original scale, the gradient, the check of the Hessian in `sdreport()`, a refit and a slice. When the end of a search depends on its starting values, multiple starts within a box of plausible values show which minima the model has and how often each is reached; their agreement is evidence, not proof, that the lowest one is the global minimum. A parameter that the likelihood cannot determine is not identifiable, structurally or practically, and an estimate at a boundary lies at $\pm\infty$ on the optimised scale; both are reported as what they are, or the model or the design is changed. Poorly scaled or strongly correlated parameters make the search harder, and a transformation, a change of units or another parameterisation can help. When BFGS keeps failing, `nlminb()` and Nelder–Mead search differently, `optimize()` handles a

single parameter, and differential evolution searches a whole box, followed by BFGS from its best member. On the diagram of slide 3, the lecture opened the entry "optimise" of the estimation box.

Return to this slide for the two closing questions.

Discussion and expected answers — allow about two minutes

Question 1: report the lowest NLL only after restarting BFGS from it and diagnosing it, and report how it was found: the box, the seed, the number of starts and how many reached the minimum, and where the others stopped. Three of 40 says that the minimum is hard to reach from this box, so the evidence that it is the global minimum is weaker than with many agreeing starts: check that the three agree on the estimates, and consider more starts, `nlminb()` or a global search (slides 14, 23 and 27). Accept answers that say the other 37 fits are worth looking at, because they show the ridges or boundaries of the model.

Question 2: there is no single correct answer, because no check is sufficient alone (slide 10). Accept the check of the Hessian in `sdreport()` (it is cheap, it is needed for the standard errors anyway, and it revealed the constant curve, the curve at zero and the stop near $c = 90$ µL/L, which the gradient did not), or fits from other starting values (they reveal a wrong minimum that every check at one point passes). The answer to push back on is the gradient alone, or the convergence code.

Final transition

This chapter completes the course's line on estimation: the sampling distributions of Chapter 3, the likelihood of Chapter 4, the response curves of Chapter 6 and the marginal likelihood of Chapter 7 all depend on an optimiser that finds their maximum, and every estimate they report assumes that it did. The formula-based tools of the supplement "Formula-based methods" rely on the same optimisers (glmmTMB uses `nlminb()`), and the same diagnosis applies to their fits. The practical comes next.

Adjustments during delivery

The slides take 116 minutes and leave 14 minutes of slack, which comes last. The demo is kept whole; if it overruns, it takes the slack first. If discussion overruns, use the slack first, then shorten in this order:

- Slide 3 to one sentence: recovers about half a minute.
- Slide 6 to the definitions, without the schematic walkthrough: about one minute.
- Slide 8 to the takeaway, with one sentence on the cost of finite differences: about one and a half minutes.
- Slide 15 to one sentence (the same failures, and slower fits): about two minutes.
- Slide 20 to the definition and the example box: about one minute.
- Slide 24 to one sentence: about one and a half minutes.
- Slide 25 to the takeaway: about two minutes.
- Slide 21 given as homework, with the answers in one sentence: about two and a half minutes.
- Slides 29 and 30 to one sentence each: about three minutes together.

Keep the fit that stops in the wrong place (slide 5), how a gradient method searches (slide 7), the stopping rule (slide 9), the diagnosis (slide 10), multiple starting values (slide 14), identifiability (slide 16), boundaries (slide 17) and global search followed by local refinement (slide 27).

The breaks are placed between the blocks, after slides 12 and 21 (minutes 35 and 75). If a block runs over, shorten the block rather than the following break.

If time is left: code to run

No code is run live apart from the demo, which covers slides 5 to 14. Students can follow the chunks of the chapter page at home, by section: “Exact and approximate gradients” (the fit without `gr` and the counts), “Diagnosing a fit” (the loop with the gradient, `pdHess` and the refit of each of the four fits, and the slice), “Structural non-identifiability” (the NLL with a common asymptote and free deviations, from three starts), “Practical non-identifiability” (the fit to the high concentrations and the two profiles with `TMB::tmbprofile()`), “Scaling” (the rate on its original scale and `parscale`), “Parameterisation” (the owl calls with two parameterisations), “Other local optimisers” (`nlminb()` from the three poor starts, and the comparison from 50 starts), “One parameter in an interval” (`optimize()` for the fir seedlings) and “Global search” (`DEoptim::DEoptim()` and BFGS from its best member, which needs the package DEoptim).